whitenoise -- RadioJOVE Guide

SWNA on Radio Burst Observations | wn.radiojove submodule

Framework: Bernido & Carpio-Bernido (2015) - World Scientific

SWNA on Radio Burst Observations * wn.radiojove submodule
Framework: Bernido & Carpio-Bernido (2015) * World Scientific

Overview

This guide covers applying Stochastic White Noise Analysis (SWNA) to radio burst observations recorded with a RadioJOVE spectrograph. The stochastic variable is the Z-score normalized intensity of a selected burst region, sampled at a fixed cadence (typically 0.1 s or 1.0 s). The memory parameter mu extracted by SWNA quantifies the dynamical memory structure of the burst.

The full pipeline has two stages:

Stage 1 -- Preprocessing (manual, using the RadioJOVE GUI):

JPG spectrograph
-> region selection GUI
-> Z-score export
-> CSV file

Stage 2 -- SWNA analysis (automated, using this package):

read_radiojove_csv()
-> [preprocess]
-> analyze_burst()
plots + CSV
-> compare_bursts()

Use analyze_burst() for a single event or batch_analyze_bursts() to process an entire folder at once. All functions in Stage 2 are accessible via wn.radiojove.*.

---

1 * Setup

```
import whitenoise as wn

# Verify the submodule is available
print(wn.__version__)
print(dir(wn.radiojove))
```

Path note: If whitenoise is installed via pip install -e . from the package root, no path configuration is needed. If running from a project that embeds the package, add to the top of your script:

```
import sys, os
sys.path.insert(0, 'path/to/whitenoise')
```

2 * Stage 1 -- Preprocessing: From Spectrograph to CSV

This stage is performed manually using the RadioJOVE observation software and a region-selection GUI. The output is one or more CSV files ready for SWNA analysis.

Step 1 -- Record a spectrum

A RadioJOVE receiver (dipole antenna + receiver + sound card) records Jupiter or solar radio emissions. The observation software produces a JPG spectrograph: time on the x-axis, frequency on the y-axis, intensity as colour. Typical observing windows are 5-30 minutes.

Step 2 -- Select burst regions (GUI)

In the region-selection tool:

- Load the JPG spectrograph
- Draw a bounding box around each burst event of interest
- The tool extracts the intensity column summed (or averaged) over the selected frequency band
- Export each region as a Z-score normalized CSV at the desired cadence

The GUI exports at two standard cadences:

Cadence	Points per second	Typical use
0.1 s	10	Fine temporal structure; higher noise
1.0 s	1	Smoothed envelope; lower noise

> Why two cadences? Running SWNA on both and comparing mu values confirms that the memory structure is robust to temporal resolution. Consistent mu at 0.1 s and 1.0 s is a sign of a genuine physical signal, not sampling artefact.

Step 3 -- Resulting CSV format

Each exported CSV has exactly two columns:

```
time_seconds,intensity_zscore
0.000,-4.050984
0.100,-4.054093
0.200,-4.057201
...
```

Column 1 is time in seconds; Column 2 is the Z-score normalized intensity (mean = 0, std = 1 across the selected burst window).

Step 4 -- Filename convention

The GUI produces filenames following the pattern:

```
YYMMDDHHMMSS<Station>_region_<N>_<cadence>s.csv

Example:
  230728154000Higgins_Home_region_1_0.1s.csv   ? 0.1 s cadence
  230728154000Higgins_Home_region_1_1.0s.csv   ? 1.0 s cadence
```

Field	Description	Example
YYMMDDHHMMSS	12-digit recording timestamp	230728154000
<Station>	Observer name or station ID	Higgins_Home
region_<N>	Region index within the spectrograph	region_1
<cadence>s	Time cadence in seconds	0.1s, 1.0s

> Z-score normalization: The GUI applies Z-score normalization automatically within each selected region. This means the exported intensity values have mean = 0 and std = 1. Do not call wn.radiojove.zscore_normalize() again on data exported by the GUI -- it is already normalized. Only use that function if you are starting from raw ADC counts or linear intensity values.

3 * Reading and Organizing Data

3.1 * List all CSVs in a folder

```
wn.radiojove.list_burst_csvs(folder, pattern='*.csv')  ->  list[str]
```

```
paths = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')
print(len(paths), 'files found')
for p in paths:
    print(p)
```

3.2 * Parse metadata from a filename

```
wn.radiojove.parse_filename_metadata(filename)  ->  dict
```

```
meta = wn.radiojove.parse_filename_metadata(
    '230728154000Higgins_Home_region_1_0.1s.csv'
)
print(meta['datetime'])    # '230728154000'
print(meta['station'])     # 'Higgins_Home'
print(meta['region'])      # 1
print(meta['cadence_s'])   # 0.1
```

Any field that cannot be parsed is returned as None -- no exception is raised.

3.3 * Read a single CSV

```
wn.radiojove.read_radiojove_csv(path)  ->  (time, intensity, metadata)
```

```
time, intensity, meta = wn.radiojove.read_radiojove_csv(
    '230728154000Higgins_Home_region_1_0.1s.csv'
)
print(f"N points : {len(time)}")
print(f"Duration : {time[-1] - time[0]:.1f} s")
print(f"Station  : {meta['station']}")
print(f"Cadence  : {meta['cadence_s']} s")
```

This wraps wn.read_csv() and enriches the returned metadata with
station, region, cadence_s, and datetime
parsed from the filename.

Metadata key	Type	Description
station	str	Station name from filename
region	int	Region index from filename
cadence_s	float	Time cadence in seconds
datetime	str	12-digit timestamp from filename
x_name	str	'time_seconds'
y_name	str	'intensity_zscore'
n_points	int	Number of rows in the CSV
source_file	str	Full path to the CSV

3.4 * Group files by cadence

```
wn.radiojove.group_by_cadence(paths)  ->  dict[float, list[str]]
```

```
paths = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')
groups = wn.radiojove.group_by_cadence(paths)

paths_01s = groups[0.1]   # all 0.1-second cadence files
paths_10s = groups[1.0]   # all 1.0-second cadence files
print(list(groups.keys()))  # [0.1, 1.0]
```

4 * Stage 2 -- Preprocessing Utilities

These functions are optional. Data exported by the RadioJOVE GUI is already Z-scored and does not require additional preprocessing before SWNA. Use these utilities only when working with raw data or when comparing analyses across different cadences.

4.1 * Z-score normalization

```
wn.radiojove.zscore_normalize(intensity)  ->  np.ndarray
```

```
# Only needed if intensity is in raw ADC counts or linear units
# (NOT needed for GUI-exported CSVs -- they are already Z-scored)
intensity_z = wn.radiojove.zscore_normalize(raw_intensity)
print(intensity_z.mean(), intensity_z.std())  # approx 0.0 and 1.0
```

4.2 * Cadence resampling

```
wn.radiojove.resample(time, intensity, target_cadence)  ->  (time_new, intensity_new)
```

```
time, intensity, _ = wn.radiojove.read_radiojove_csv('region_1_0.1s.csv')

# Downsample from 0.1 s to 1.0 s using linear interpolation
time_1s, intensity_1s = wn.radiojove.resample(time, intensity, target_cadence=1.0)
print(f"{len(time)} -> {len(time_1s)} points")
```

> When to use resample: The GUI already exports two cadences (0.1 s and 1.0 s) separately. Use resample() only if you have a single cadence and need to generate a comparison at a different resolution, or if the exported CSV has irregular spacing that needs standardizing.

5 * Single Burst Analysis -- Step by Step

Use this approach when you want to inspect each stage individually.

Step 1 -- Read the CSV

```
time, intensity, meta = wn.radiojove.read_radiojove_csv('region_1_0.1s.csv')
```

Step 2 -- (Optional) Detrend

A linear trend within the burst window may be present due to receiver drift or the burst envelope. Removing it isolates the stochastic fluctuations used in SWNA.

```
intensity_detrended = wn.detrend(intensity, method='linear')
```

Step 3 -- Compute MSD

```
lags, msd = wn.compute_msd(intensity_detrended)
print(f"MSD: {len(lags)} lags, MSD[1]={msd[0]:.4f}, MSD[-1]={msd[-1]:.4f}")
```

> Lag range: compute_msd() computes lags 1 to N/2 by default.
> N/2 is the statistical reliability cutoff -- beyond that, each lag value is estimated from fewer than two full sweep lengths. All N/2 lags are used for fitting (the default max_lag_fraction=1.0).

Step 4 -- Fit a model

```
fit = wn.fit_msd(lags, msd, model='exponential')
if fit:
    print(fit.summary())
```

Step 5 -- Plot

```
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Linear scale
ax1.scatter(lags, msd, s=8, color='steelblue', alpha=0.7, label='Empirical MSD')
ax1.plot(fit.lags_used, fit.msd_fitted, color='crimson', lw=2, label='exponential fit')
ax1.set_xlabel('Time lag (s)')
ax1.set_ylabel('MSD')
ax1.legend()

# Log-log scale
ax2.loglog(lags, msd, '.', color='steelblue', alpha=0.7, label='Empirical MSD')
ax2.loglog(fit.lags_used, fit.msd_fitted, color='crimson', lw=2)
ax2.set_xlabel('Time lag (s)')
ax2.set_ylabel('MSD')

plt.tight_layout()
plt.savefig('burst_msd.png', dpi=150)
plt.show()
```

6 * Full Single-Burst Pipeline (One Call)

```
wn.radiojove.analyze_burst(path, model='exponential', detrend_method='linear',
                           normalize=False, output_dir=None, verbose=True)  ->  dict
```

Runs all five steps above in a single call and returns a dict with the result, figure, and summary row.

```
out = wn.radiojove.analyze_burst(
    '230728154000Higgins_Home_region_1_0.1s.csv',
    model='exponential',
    detrend_method='linear',
    output_dir='swna_results/',   # saves plot PNG here
)

result = out['result']
if result and result.fit:
    print(f"mu    = {result.fit.params['mu']:.4f}")
    print(f"R2    = {result.fit.r_squared:.4f}")
    print(f"regime: {result.regime}")

out['figure'].show()     # dual-panel MSD plot (linear + log-log)
```

Return key	Type	Description
path	str	Input file path
dataset	str	Filename stem (no extension)
result	AnalysisResult	Full SWNA result (None if failed)
figure	Figure	Dual-panel MSD plot
row	dict	Summary row for DataFrame construction

> detrend_method='linear' is the recommended default. It removes a linear trend within the burst window, isolating stochastic fluctuations from receiver drift or the burst envelope. Set to None to skip detrending.

Plotting a result separately

```
wn.radiojove.plot_burst_msd(result, title='', save_path=None)  ->  Figure
```

```
fig = wn.radiojove.plot_burst_msd(
    out['result'],
    title='Region 1 - 0.1s cadence',
    save_path='region1_msd.png',
)
fig.show()
```

7 * Batch Analysis

```
wn.radiojove.batch_analyze_bursts(source, model='exponential',
                                  detrend_method='linear', normalize=False,
                                  output_dir=None, n_jobs=1, verbose=True)
                                  ->  pd.DataFrame
```

Processes all CSVs in a folder (or an explicit list), saves per-file MSD plots, and returns a summary DataFrame with one row per file.

```
df = wn.radiojove.batch_analyze_bursts(
    'Solar/Type 3 Bursts/first trials/',
    model='exponential',
    output_dir='swna_results/',
)

print(df[['dataset', 'mu', 'mu_ci_low', 'mu_ci_high', 'r_squared', 'regime']])
```

You can also pass a specific list of files:

```
paths_01s = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')
paths_01s = [p for p in paths_01s if '0.1s' in p]

df = wn.radiojove.batch_analyze_bursts(paths_01s, model='exponential')
```

DataFrame column	Description
dataset	Filename stem
filename	Full filename with extension
model	SWNA model used
n_points	Number of time points in the burst
mu	Memory parameter mu
mu_ci_low	95% CI lower bound for mu
mu_ci_high	95% CI upper bound for mu
beta	Exponential decay rate beta (exponential
r_squared	Goodness of fit R2
regime	Diffusion regime (see Section 8)

> Parallel processing: Set n_jobs=4 (or more) to analyze files in parallel using threads. I/O-bound analysis on large folders is typically 2-4x faster with parallelism.

8 * Comparing Bursts

```
wn.radiojove.compare_bursts(paths, model='exponential',
                            detrend_method='linear', normalize=False,
                            max_lag_fraction=1.0)  ->  ComparisonResult
```

```
# Group files by cadence and compare only the 0.1 s files
paths = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')
groups = wn.radiojove.group_by_cadence(paths)

cr = wn.radiojove.compare_bursts(groups[0.1], model='exponential')

# ASCII table
wn.print_comparison(cr)

# Publication-quality mu bar chart
wn.publish_comparison(cr)
```

The returned ComparisonResult also contains:

```
print(cr.summary_df)        # full results DataFrame
cr.summary_df.to_csv('comparison.csv', index=False)
```

9 * Multi-Resolution Comparison (0.1 s vs 1.0 s)

Comparing mu values at two cadences verifies that the memory structure is resolution-independent. Robust results should show consistent mu at both cadences (within confidence intervals).

```
import pandas as pd

paths = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')
groups = wn.radiojove.group_by_cadence(paths)

df_01s = wn.radiojove.batch_analyze_bursts(groups[0.1], model='exponential', verbose=False)
df_10s = wn.radiojove.batch_analyze_bursts(groups[1.0], model='exponential', verbose=False)

df_01s['cadence'] = '0.1s'
df_10s['cadence'] = '1.0s'

df_all = pd.concat([df_01s, df_10s], ignore_index=True)
print(df_all[['dataset', 'cadence', 'mu', 'r_squared', 'regime']])
```

> Interpretation: If mu at 0.1 s and 1.0 s agree to within their 95% confidence intervals, the memory structure is robust. A significant discrepancy suggests the result may be influenced by noise at the finer cadence or by smoothing artefacts at the coarser one.

10 * The Exponential SWNA Model

```
MSD(T, mu, beta) = Gamma(mu) * beta^(-mu) * T^(mu-1) * exp(-beta / T)
```

Parameter	Role	Typical range (Type III bursts)
mu (mu)	Memory exponent -- controls long-time MS	1.4-1.7
beta (beta)	Exponential decay rate -- controls where	10-100
N	Normalization scalar -- fitted automatic	?1

> Why exponential and not cosine? The cosine model produces an oscillating MSD that can become negative, causing catastrophic failure (R2 approx -1060) on data with a monotonically growing MSD. Type III solar bursts and most Jovian S-bursts have monotonically growing MSDs -- always use model='exponential' for these event types. Use wn.list_models() to see all available models.

Interpreting mu

mu range	Regime	Physical interpretation
mu < 0.95	subdiffusive	Anti-correlated fluctuations; restricted
0.95 <= mu <= 1.05	near-Brownian	Weak or no memory; ordinary random walk
1.05 < mu <= 2.0	superdiffusive	Positively correlated; fast spreading
mu > 2.0	hyperballistic	Strongly persistent; extreme memory

Type III solar bursts are driven by semi-relativistic electron beams spiralling along open coronal magnetic field lines. These beams produce positively correlated, fast-spreading intensity fluctuations -- consistent with the observed mu approx 1.4-1.6 (superdiffusive) regime.

11 * When the Fit is Poor

Symptom: R2 < 0.5 or very negative

First check the model choice. The cosine model fails catastrophically on monotonic MSD data (R2 can be -1060). Switch to model='exponential'.

```
out = wn.radiojove.analyze_burst('burst.csv', model='exponential')
```

Symptom: fit curve covers only part of the MSD points

This can happen if max_lag_fraction is set below 1.0. The default is 1.0, which fits all displayed MSD lags. Do not change this unless you are deliberately excluding the noisy high-lag tail.

Symptom: good shape but low R2

The optimizer found a local minimum. Try providing custom initial parameters:

```
import whitenoise as wn

time, intensity, _ = wn.radiojove.read_radiojove_csv('burst.csv')
intensity_d = wn.detrend(intensity, method='linear')
lags, msd   = wn.compute_msd(intensity_d)

# Inspect the MSD visually first to estimate mu
fit = wn.fit_msd(lags, msd, model='exponential',
                 p0=[1.5, 20.0],   # [mu_guess, beta_guess]
                 bounds=([0.1, 0.001], [4.0, 500.0]))
print(fit.summary())
```

Symptom: burst is very short (< 50 time points)

The MSD computed from a short burst has very few lags and high statistical noise.
Consider using the 0.1 s cadence file (10x more points for the same event).
If the burst is genuinely short, report the result with a low confidence note.

12 * Complete Workflow Example

```
import whitenoise as wn
import pandas as pd
import os

# ?? Configuration ????????????????????????????????????????????????????????
DATA_DIR   = 'Solar/Type 3 Bursts/first trials/'
OUT_DIR    = 'swna_results/'
MODEL      = 'exponential'
os.makedirs(OUT_DIR, exist_ok=True)

# ?? Step 1: List and organize files ??????????????????????????????????????
paths  = wn.radiojove.list_burst_csvs(DATA_DIR)
groups = wn.radiojove.group_by_cadence(paths)

print(f"Found {len(paths)} files:")
for cadence, fpaths in sorted(groups.items(), key=lambda x: x[0] or 0):
    print(f"  {cadence}s: {len(fpaths)} file(s)")

# ?? Step 2: Batch analyze at both cadences ???????????????????????????????
df_01s = wn.radiojove.batch_analyze_bursts(
    groups[0.1], model=MODEL,
    output_dir=os.path.join(OUT_DIR, '01s'),
    verbose=True,
)
df_10s = wn.radiojove.batch_analyze_bursts(
    groups[1.0], model=MODEL,
    output_dir=os.path.join(OUT_DIR, '10s'),
    verbose=False,
)

# ?? Step 3: Merge and save summary ???????????????????????????????????????
df_01s['cadence'] = '0.1s'
df_10s['cadence'] = '1.0s'
df_all = pd.concat([df_01s, df_10s], ignore_index=True)
df_all.to_csv(os.path.join(OUT_DIR, 'swna_summary.csv'), index=False)

cols = ['dataset', 'cadence', 'mu', 'mu_ci_low', 'mu_ci_high', 'r_squared', 'regime']
print(df_all[cols].to_string(index=False))

# ?? Step 4: Compare 0.1s cadence bursts ??????????????????????????????????
cr = wn.radiojove.compare_bursts(groups[0.1], model=MODEL)
wn.print_comparison(cr)

# Publication-quality mu comparison bar chart
fig = wn.publish_comparison(cr, show=False)
fig.savefig(os.path.join(OUT_DIR, 'comparison_mu.png'), dpi=150, bbox_inches='tight')
```

13 * Function Reference

Function	Location	Description
list_burst_csvs(folder)	wn.radiojove	List all CSVs in a folder, sorted
parse_filename_metadata(filename)	wn.radiojove	Extract datetime, station, region, caden

read_radiojove_csv(path)	wn.radiojove	Read CSV; returns (time, intensity, enri
group_by_cadence(paths)	wn.radiojove	Group file list by time cadence
zscore_normalize(intensity)	wn.radiojove	Z-score normalize (only if not already d
resample(time, intensity, cadence)	wn.radiojove	Resample to a different cadence
analyze_burst(path, model, ...)	wn.radiojove	Full SWNA pipeline for one burst CSV
plot_burst_msd(result, ...)	wn.radiojove	Dual-panel MSD plot (linear + log-log)
batch_analyze_bursts(source, ...)	wn.radiojove	Batch analysis; returns summary DataFram
compare_bursts(paths, model, ...)	wn.radiojove	Multi-burst comparison via wn.compare()
compute_msd(x)	wn	Empirical MSD (lags 1 to N/2)
fit_msd(lags, msd, model)	wn	Fit any SWNA model; returns FitResult
detrend(values, method)	wn	Remove linear/polynomial/mean trend
list_models()	wn	Print all 16 SWNA models and status
print_comparison(cr)	wn	ASCII comparison table
publish_comparison(cr)	wn	Publication-quality mu bar chart

whitenoise * RadioJOVE SWNA Guide *

Framework: Bernido C C, Carpio-Bernido M V (2015). Methods and Applications of White Noise Analysis in Interdisciplinary Sciences. World Scientific.